Ecole Supérieure d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

- TP 07: Cryptography
- Coordinator: Prof. Anass Sebbar
- Level: 4th year Eng

TP n° 07

Cryptography

1- Objectives:

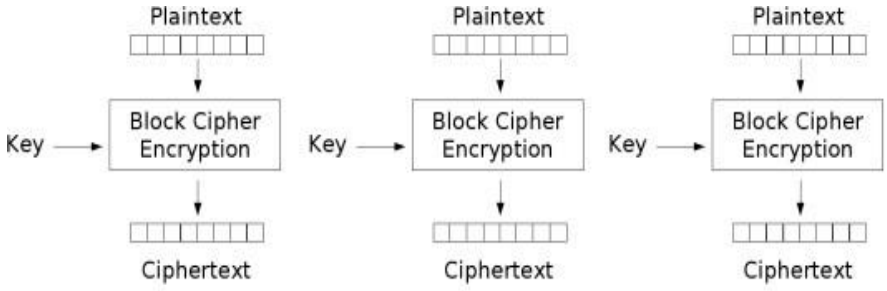
Cryptography is the science of encoding messages. Indeed, it is common for a person (let's call her Alice) to try to communicate with a second person (named Bob) without a third (carl) pissing to read the lessons sent. There is a wide variety of ways to achieve this coding, the most efficient of which call for high-flying mathematics.
The objective of this lab is to understand the different encryption systems and to do a vertical analysis of encryption and decryption of messages, and finally cryptoanalysis.

We'll look at:

- Operating Procedures
- Feistel schema implementation

Exercise 1: ECB

We consider a block cryptosystem that applies a permutation to binary vectors of size 4 in ECB mode.



Electronic Codebook (ECB) mode encryption

The permutation function π is defined as follows:

$b_1b_2b_3b_4 \rightarrow b_{\pi(1)}b_{\pi(2)}b_{\pi(3)}b_{\pi(4)}$

On donne l'operation de permutation :

$$\pi = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 3 & 4 & 1 \end{pmatrix}$$

Let the plaintext m be :

m = 101100010100101

1) Describe mathematically or with pseudo-code how ECB works.
2) Break down the plaintext into blocks of appropriate size n=4. Jamming with zeros to get blocks of the same size
3) Apply ECB mode when encrypting plaintext blocks
4) Donner le ciphertext final
5) Apply decryption and verify with the original message
6) Considering a plaintext formed by the same 1010 blocks, is this redundancy propagated in the ciphertext?
7) What if the order of the ciphertexts blocks is changed? Is it possible to decrypt each block?
8) What do you think of the security of ECB and in which application is it appropriate?

Exercise 2: CBC

We use my same key, the same plaintext, the same operation but in CBC mode.

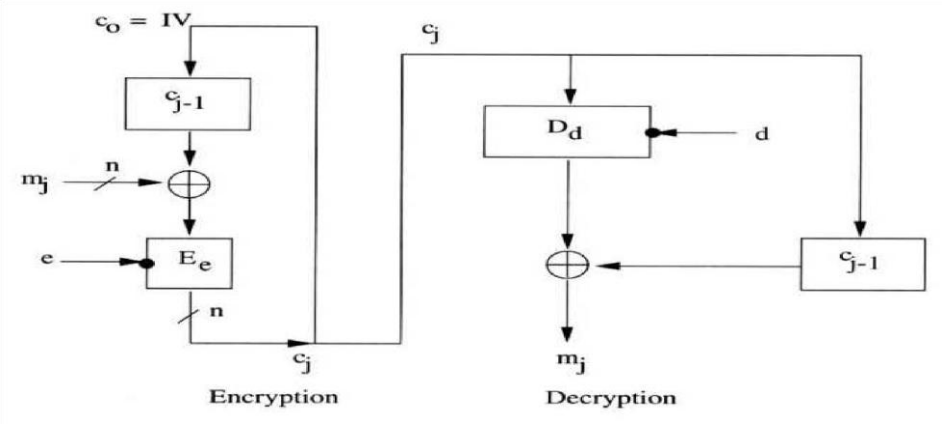


Fig : CBC mode

The permutation function π is

defined as follows:

$b_1b_2b_3b_4 \rightarrow b_{\pi(1)}b_{\pi(2)}b_{\pi(3)}b_{\pi(4)}$

On donne l'operation de permutation :

$$\pi = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 3 & 4 & 1 \end{pmatrix}$$

Let the plaintext m be :
m = 101100010100101

1) Describe mathematically or with pseudo-code how CBC works.
2) Break down the plaintext into blocks of appropriate size. Jamming with zeros to get blocks of the same size
3) Apply CBC mode when encrypting plaintext blocks
4) Donner le ciphertext final
5) Apply decryption and verify with the original message
6) Considering a plaintext formed by the same 1011 blocks, is this redundancy propagated in the ciphertext?
7) What if the order of the ciphertexts blocks is changed? Is it possible to decrypt each block?
8) What did you think of CBC's security and in which application is it appropriate?
9) If an error occurs in the first block of the ciphertext. Study the propagation of error on decryption (say which blocks are affected and which blocks are intact in the plaintext).
10) Saying which CBC app is appropriate

Exercise 3:

A Fiscel scheme is an iterative encryption system: it is divided into several towers or floors and encryption and decryption have a similar architecture.

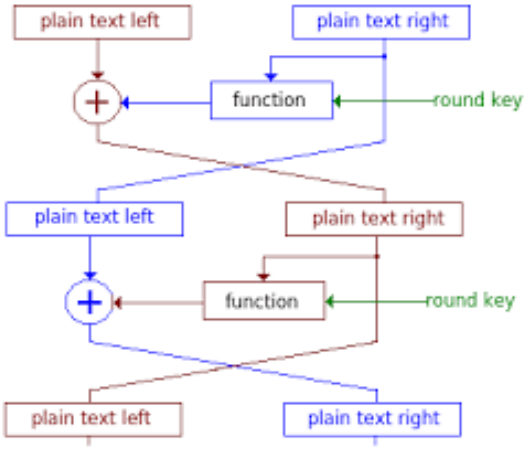


Fig 3: Feistel diagram

Developer a Feistel encryption algorithm (Python – jupter notebook)

1. Create a list of all characters in plain text.
2. Convert plain text to Ascii and then to 8-bit binary format.
3. Divide the plain text binary string into two halves: left half (L1) and right half (R1)
4. Generate a random binary key (K1 and K2) with a length equal to half the length of the plain text for both rounds.

- **1er Round of Encryption**

```
a. Generate function f1 using R1 and K1 as follows:
                f1= xor(R1, K1)
b. Now the new left half(L2) and right half(R2) after
round 1 are as follows:
                R2= xor(f1, L1)
                L2=R1
```

- **2eme Round of Encryption**

```
a. Generate function f2 using R2 and K2 as follows:
                f2= xor(R2, K2)
b. Now the new left half(L2) and right half(R2) after
round 1 are as follows:
                R3= xor(f2, L2)
                L3=R2
```

5. Decryption applies the same keys in reverse order (`K2` followed by `K1`) to recover the plaintext.